Department: 資工系

Student ID: 112550081

Name: 許秉棋

Problem 1.

Ciphertext:

GYOMXNOGNG QUGN ETNMX MPLMZOMXYM K TMMJOXA XEN TKZ ZMQEBMF TZEQ KJKZQ EX YEXNMQLJKNOXA NHM TJEEF ET XMI CXEIJMFAM IHOYH MKYH WMKZ RZOXAG IONH ON

a (112550081_1_a.py).

Letter	Count	Frequency
M	17	14.0%
X	11	9.1%
N	10	8.3%
E	10	8.3%
O	9	7.4%
Z	7	5.8%
K	7	5.8%
T	6	5.0%
G	5	4.1%
Y	5	4.1%
Q	5	4.1%
J	5	4.1%
H	5	4.1%
A	4	3.3%
I	4	3.3%
F	3	2.5%
L	2	1.7%
U	1	0.8%
P	1	0.8%
B	1	0.8%
C	1	0.8%
W	1	0.8%
R	1	0.8%

b.

Using the standard English letter frequency distribution as a reference, we align the most frequent letters in the ciphertext with those in English:

The most frequent letter in the ciphertext is M (14.0%), which likely corresponds to E (12.7%) in English plaintext. The second most frequent is X (9.1%), suggesting it maps to T (9.1%). Similarly, N and E in the ciphertext (both ~8.3%) are candidates for A and O respectively.

So we might decrypt:
"SCIENTISTSMUSTOFTENEXPERIENCEAFEELINGNOTFARREMOVEDFROMALARMONCONTEMPLATINGTHEFLOODOFNEWKNOWLEDGEWHICHEACHYEARBRINGSWITHIT"

c.

From part (b), the most frequent letter in the ciphertext is M, which we expect corresponds to E in English plaintext. By testing various structured cipher models, we find that a consistent affine (linear) relationship holds across all letters:

$$C \equiv aP + b \ (mod\ 26)$$

To verify, we check several plaintext–ciphertext pairs (confirmed by full decryption):

Plaintext P	P value	C value	Ciphertext C
E	4	12	M
S	18	6	G
I	8	14	O
C	2	24	Y

Every letter satisfies the same linear formula with the same constants. Therefore, a specific mathematical relationship does exist: the encryption is an affine cipher, where each plaintext letter P is mapped to ciphertext C via a fixed linear function modulo 26.

d.

To determine the values of a and b, we use two known plaintext–ciphertext pairs

to set up a system of equations: E->M: P=4, C=12 and S->G: P=18, C=6. Then substituting into $C \equiv aP + b \ (mod\ 26)$:

$$4a + b \equiv 12 \ (mod\ 26) \quad (1)$$
$$18a + b \equiv 6 \ (mod\ 26) \quad (2)$$

Subtracting (1) from (2): $14a \equiv -6 \ \equiv 20 \ (mod\ 26)$ and then dividing both sides by 2: $7a \equiv 10 \ (mod\ 13)$. Since $7 \times 2 \equiv 14 \equiv 1 \ (mod\ 13)$, the inverse of 7 mod 13 is 2. Thus: $a \equiv 2 \times 10 = 20 \equiv 7 \ (mod\ 13) \rightarrow a = 7.$

Substituting a = 7 back into (1):

$$4(7) + b \equiv 12 \ (mod\ 26)$$

$$28 + b \equiv 12 \ (mod\ 26)$$

$$b \equiv 12 - 2 = 10 \ (mod\ 26) \rightarrow b = 10$$

And we apply $f(x) = 7x + 10 \ (mod\ 26)$ to the full plaintext for verification:

Plaintext	P	7P+10 mod 26	C	Ciphertext
S	18	136 mod 26 = 6	G	y
E	4	38 mod 26 = 12	M	y
I	8	66 mod 26 = 14	O	y
N	13	101 mod 26 = 23	X	y

So the value for a and b are: $a = 7, b = 10.$

e.

In a monoalphabetic substitution cipher, the key is a permutation of the 26 letters of the alphabet, each plaintext letter is mapped to a unique ciphertext letter. The total number of possible keys is therefore:

$$|K| = 26! = 403{,}291{,}461{,}126{,}605{,}635{,}584{,}000{,}000 \approx 4 \times 10^{26}$$

While this key space is astronomically large that far beyond the reach of any brute-force search, it's not secure in practice. The reason is that it preserves the statistical structure of the plaintext: each plaintext letter always maps to the same ciphertext letter, so the frequency distribution of letters is unchanged. As demonstrated in part (b), frequency analysis can break this cipher with only moderate amount of ciphertext, regardless of how large the key space is.

f.

For $f(x) = ax + b \ (mod\ 26)$, the key is the pair (a, b) subject to the constraint that $\gcd(a, 26) = 1$ (so that the function is a valid bijection and decryption is possible. **Valid values of a**: must satisfy $\gcd(a, 26) = 1$. Since $26 = 2 \times 13$, we exclude multiples of 2 and 13:

$$a \in \{1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25\} \rightarrow 12\ choices$$

Valid values of b: any integer from 0 to 25:

$$b \in \{0, 1, \dots, 25\} \rightarrow 26\ choices$$

$$|K|_{affine} = 12 \times 26 = 312$$

This is drastically smaller than $26! \approx 4 \times 10^{26}$, making exhaustive search trivial (as shown in part (g)). And one of the approaches to increase the key space might apply the affine cipher multiple times with different keys, i.e., compose several affine transformations:

$$f_1\big(f_2(x)\big) = a_1(a_2x + b_2) + b_1 = (a_1a_2)x + (a_1b_2 + b_1)\ (mod\ 26)$$

However, the composition of two affine ciphers is still an affine cipher, so this does not increase the effective key space. A more effective approach is to use a polyalphabetic substitution cipher (e.g., the Vigenère cipher), where a different affine (or shift) key is applied to each position in the plaintext, breaking the fixed letter-to-letter mapping and making frequency analysis much harder.

g (112550081_1_g.py).

```python
import math

# English letter frequencies (%)
ENGLISH_FREQ = {
    'E':12.7,'T':9.1,'A':8.2,'O':7.5,'I':7.0,'N':6.7,'S':6.3,'H':6.1,
    'R':6.0,'D':4.3,'L':4.0,'C':2.8,'U':2.8,'M':2.4,'W':2.4,'F':2.2,
    'G':2.0,'Y':2.0,'P':1.9,'B':1.5,'V':1.0,'K':0.8,'J':0.2,'X':0.2,
    'Q':0.1,'Z':0.1
}

def mod_inverse(a, m=26):
    # Step 1: Find a^-1 mod m by brute force
    for x in range(1, m):
        if (a * x) % m == 1:
            return x
    return None

def affine_decrypt(ciphertext, a, b):
    # Step 2: Decrypt using P = a_inv * (C - b) mod 26
    a_inv = mod_inverse(a)
    result = ''
    for c in ciphertext:
        p = (a_inv * (ord(c) - ord('A') - b)) % 26
        result += chr(p + ord('A'))
    return result

def score_text(text):
    # Step 3: Score by summing English letter frequencies
    return sum(ENGLISH_FREQ.get(ch, 0) for ch in text)

# Ciphertext (letters only, uppercase)
ciphertext_raw = """GYOMXNOGNG QUGN ETNMX MPLMZOMXYM K TMMJOXA XEN TKZ ZMQEBMF TZEQ
KJKZQ EX YEXNMQLJKNOXA NHM TJEEF ET XMI CXEIJMFAM IHOYH
MKYH WMKZ RZOXAG IONH ON"""
ciphertext = ''.join(c for c in ciphertext_raw if c.isalpha()).upper()

# Step 4: Find all valid values of a where gcd(a, 26) = 1
valid_a = [a for a in range(1, 26) if math.gcd(a, 26) == 1]
print(f"Step 1: Valid values of a: {valid_a}")
print(f"Step 2: Try all b in 0..25 → total {len(valid_a)*26} keys\n")

# Step 5: Try all valid (a, b) pairs and score each decryption
candidates = []
for a in valid_a:
    for b in range(26):
        decrypted = affine_decrypt(ciphertext, a, b)
        s = score_text(decrypted)
        candidates.append((s, a, b, decrypted))

# Step 6: Sort by score descending and print top candidates
candidates.sort(reverse=True)
print("Top 3 candidates:\n" + "-"*60)
for i, (s, a, b, dec) in enumerate(candidates[:3]):
    print(f"Rank {i+1}: a={a}, b={b}, score={s:.1f}")
    print(f"  {dec}\n")
```

Step 1: Identify All Valid Values of $f(x) = ax + b\ (mod\ 26)$ to be a valid encryption function, it must be a bijection (one-to-one mapping), every plaintext letter must map to a unique ciphertext letter. This requires that the decryption function $f^{-1}(x)$ exist, which is only possible when a has a modular inverse mod 26. The condition is: $\gcd(a, 26) = 1$. Since $26 = 2 \times 13$, we must exclude any a that is divisible by 2 or 13. This gives exactly 12 valid values we mentioned in part e.

Step 2: Enumerate All Key Pairs (a, b). The shift parameter b can be any integer from 0 to 25 (26 choices), with no additional constraint, the total keys to try is 312 keys.

Step 3: For each candidate key (a, b), we compute the modular inverse $a^{-1}(mod\ 26)$ and apply the decryption formula to every letter in the ciphertext:

$$P = a^{-1} \cdot (C - b)\ (mod\ 26)$$

For example, with $a = 7, b = 10,\ a^{-1} = 15$ since $7 \times 15 = 105 \equiv 1\ (mod\ 26)$,
For $C = M = 12$: $P = 15 \times (12 - 10) = 30 \equiv 4 = E$

Step 4: Score Each Decryption by English Letter Frequency. Not all 312 decryptions produce readable English. We assign a score to each candidate plain text by summing the standard English frequencies of letter it contains:

$$score = \sum_i freq_english(P_i)$$

A decryption that closely matches the natural distribution of English letters (e.g., contains many E's, T's, A's) will receive a **higher score**, while random-looking text will score low. This allows us to automatically rank all candidates without human inspection.

Step 5: Rank and Output the Best Candidates. After scoring all 312 keys, we sort them in **descending order** by score and print the top results:

Rank	a	b	score	plaintext
1	7	10	781.8	SCIENTISTSMUSTOFTENEXPERIENCEAFEELINGNOTFARRE MOVEDFROMALARMONCONTEMPLATINGTHEFLOODOF NEWKNOWLEDGEWHICHEACHYEARBRINGSWITHIT

2	25	16	634.4	KSCETDCKDKAWKDMXDETEBFERCETSEGXEEHCTQTMDXG RREAMPELXRMAGHGRAMTSMTDEAFHGDCTQDJEXHMMLM XTEIOTMIHELQEIJCSJEGSJUEGRZRCTQKICDJCD
3	1	19	629.3	NFVTEUVNUNXBNULAUTETWSTGVTEFTRATTQVEHELUARGGT XLITMAGLXRQRGXLEFLEUTXSQRUVEHUOTAQLLMLAETPJELP QTMHTPOVFOTRFODTRGYGVEHNPVUOVU

The **Rank 1** candidate is clearly readable English, confirming: $a = 7, b = 10$ with the recovered plaintext:

"Scientists must often experience a feeling not far removed from alarm on contemplating the flood of new knowledge which each year brings with it."

Problem 2.

Step 1: Determine the Rectangle Dimensions. The contains **63 letters**. Since the message completely fills the rectangle, we find all factor pairs of 63: $63 = 7 \times 9 \; or \; 9 \times 7$. This gives two candidate dimensions: a **7-column × 9-row** or a **9-column × 7-row** rectangle.

Step 2: Use Vowel Frequency to Select the Correct Dimension. In English plaintext, approximately **40% of letters are vowels** (A, E, I, O, U). For the correct rectangle dimension, each **row** of the reconstructed plaintext should contain roughly 40% vowels. We test both cases.

Case 1: **9 columns × 7 rows** — each row has 9 letters, expected vowels per row: $0.4 \times 9 = 3.6$ Arrange the ciphertext into 9 columns (each column has 7 letters), then read row by row:

E	C	D	T	M	E	C	A	E
R	A	U	O	O	L	E	D	S
A	M	M	E	R	N	E	N	A
S	S	O	D	Y	T	N	R	V
B	N	L	C	R	L	T	I	Q
L	A	E	T	R	I	G	A	W
E	B	A	A	E	I	H	O	R

Row	Letters	Vowels	Difference from 3.6
1	ECDTMECAE	4	0.4
2	RAUOOLEDS	5	1.4
3	AMMERNENA	4	0.4
4	SSODYTNRV	1	2.6
5	BNLCRLTIQ	1	2.6
6	LAETRIGAW	3	0.6
7	EBAAEIHOR	6	2.4

Total difference: 0.4+1.4+0.4+2.6+2.6+0.6+2.4=10.2

Case 2:

7 columns × 9 rows — each row has 7 letters, expected vowels per row: $0.4 \times 7 = 2.8$, arrange the ciphertext into 7 columns (each column has 9 letters), then read row by row.

E	C	D	T	M	E	C
A	E	R	A	U	O	O
L	E	D	S	A	M	M
E	R	N	E	N	A	S
S	O	D	Y	T	N	R
V	B	N	L	C	R	L
T	I	Q	L	A	E	T
R	I	G	A	W	E	B
A	A	E	I	H	O	R

Row	Letters	Vowels	Difference from 3.6
1	ECDTMEC	2	0.8
2	AERAUOO	6	3.2
3	LEDSAMM	2	0.8
4	ERNENAS	3	0.2
5	SODYTNR	1	1.8
6	VBNLCRL	0	2.8

7	TIQLAET	3	0.2
8	RIGAWEB	3	0.2
9	AAEIHOR	5	2.2

Total difference: $0.8 + 3.2 + 0.8 + 0.2 + 1.8 + 2.8 + 0.2 + 0.2 + 2.2 = \mathbf{12.2}$

Comparing both cases:

Dimension	Total Vowel Deviation
9 columns x 7 rows	10.2
7 columns x 9 rows	12.2

Conclusion: Since $10.2 < 12.2$, the **7 blocks of 9 letters** configuration has smaller total deviation, indicating the correct structure is a **9 rows × 7 columns** grid.

Step 4: Reconstruct the Grid. Laying out the 7 blocks of 9 letters as the columns of a 9x7 grid in case 2, we can find the correct column order. By examining the rows and anagramming them to form readable English words, we find the correct column order is $Key = (6, 3, 4, 1, 2, 5, 7)$ and then rearranging in this order row yields:

$LASER\ BEAMS\ CAN\ BE\ MODULATED\ TO\ CARRY\ MORE\ INTELLIGENCE\ THAN\ RADIO\ WAVES$

Problem 3.

Given information, the plaintext "THE" is encrypted using the following 3x3 key matrix K:

$$K = \begin{pmatrix} 6 & 24 & 1 \\ 13 & 16 & 10 \\ 20 & 17 & 15 \end{pmatrix}$$

Producing ciphertext "AJN". We must decrypt "AJN" back to "THE" using $K^{-1}\ (mod\ 26)$.

Step 1: Compute the Determinant of K

$$\det(K) = 6(16 \cdot 15 - 10 \cdot 17) - 24(13 \cdot 15 - 10 \cdot 20) + 1(13 \cdot 17 - 16 \cdot 20)$$

$$= (240 - 170) - 24(195 - 200) + 1(221 - 320)$$

$$= 6(70) - 24(-5) + 1(-99)$$

$$= 420 + 120 - 99 = 441$$

$$\det(\mathrm{K}) \equiv 441(\mathrm{mod}26) = 441 - 16 \times 26 = 441 - 416 = 25$$

Step 2: Verify the Determinant is Valid. Since $26 = 2 \times 13$, we need $\gcd(\det(K), 26) = 1$: $\gcd(25,26) = 1$. The determinant is neither even nor dividable by 13, so the inverse matrix exists.

Step 3: Compute the Modular Inverse of the Determinant. We need d^{-1} such that $25 \cdot d^{-1} \equiv 1\ (mod\ 26)$:

$$25 \times 25 = 625 = 24 \times 26 + 1 \equiv 1(\mathrm{mod}26)$$

$$\therefore (\det(\mathrm{K})) - 1 \equiv 25(\mathrm{mod}26)$$

Step 4: Compute the Adjugated Matrix. The adjugate is the transpose of the cofactor matrix. We compute each cofactor $C_{ij} = (-1)^{i+j} M_{ij}$:

$$\mathrm{C11} = +(16 \cdot 15 - 10 \cdot 17) = 240 - 170 = 70$$

$$C12 = -(13 \cdot 15 - 10 \cdot 20) = -(195 - 200) = 5$$

$$\mathrm{C13} = +(13 \cdot 17 - 16 \cdot 20) = 221 - 320 = -99$$

$$C21 = -(24 \cdot 15 - 1 \cdot 17) = -(360 - 17) = -343$$

$$\mathrm{C22} = +(6 \cdot 15 - 1 \cdot 20) = 90 - 20 = 70$$

$$\mathrm{C23} = -(6 \cdot 17 - 24 \cdot 20) = -(102 - 480) = 378$$

$$\mathrm{C31} = +(24 \cdot 10 - 1 \cdot 16) = 240 - 16 = 224$$

$$\mathrm{C32} = -(6 \cdot 10 - 1 \cdot 13) = -(60 - 13) = -47$$

$$\mathrm{C33} = +(6 \cdot 16 - 24 \cdot 13) = 96 - 312 = -216$$

Therefore the adjugate (transpose of cofactor matrix) is:

$$adj(K) = \begin{pmatrix} 70 & -343 & 224 \\ 5 & 70 & -47 \\ -99 & 378 & -216 \end{pmatrix}$$

Step 5: Compute $K^{-1}\ (mod\ 26)$. $K^{-1} \equiv 25 \cdot adj(K)\ (mod\ 26)$, reducing each entry (note: negative values use x mod 26, e.g., $-343\ mod\ 26 = 7$):

$$K^{-1} \equiv \begin{pmatrix} 25 \times 70 & 25 \times (-343) & 25 \times 224 \\ 25 \times 5 & 25 \times 70 & 25 \times (-47) \\ 25 \times (-99) & 25 \times 378 & 25 \times (-216) \end{pmatrix} (mod\ 26)$$

$$= \begin{pmatrix} 1750 & -8575 & 5600 \\ 125 & 1750 & -1175 \\ -2475 & 9450 & -5400 \end{pmatrix} (mod\ 26) = \begin{pmatrix} 8 & 5 & 10 \\ 21 & 8 & 21 \\ 21 & 12 & 8 \end{pmatrix}$$

Step 6: Verify $K \cdot K^{-1} \equiv \mathrm{I}\ (\mathrm{mod}\ 26)$

$$K \cdot K^{-1} = \begin{pmatrix} 6 & 24 & 1 \\ 13 & 16 & 10 \\ 20 & 17 & 15 \end{pmatrix} \begin{pmatrix} 8 & 5 & 10 \\ 21 & 8 & 21 \\ 21 & 12 & 8 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} (mod\ 26)\ \rightarrow correct$$

Step 7: Decrypt "AJN", convert ciphertext to integers: $A = 0,\ J = 9, N = 13$

$$p = K^{-1} \cdot c\ (mod\ 26) = \begin{pmatrix} 8 & 5 & 10 \\ 21 & 8 & 21 \\ 21 & 12 & 8 \end{pmatrix} \begin{pmatrix} 0 \\ 9 \\ 13 \end{pmatrix} (mod\ 26)$$

$$\text{Row 1: } 8(0) + 5(9) + 10(13) = 0 + 45 + 130 = 175 \equiv 19 = \mathrm{T}$$

$$\text{Row 2: } 21(0) + 8(9) + 21(13) = 0 + 72 + 273 = 345 \equiv 7 = \mathrm{H}$$

$$\text{Row 3: } 21(0) + 12(9) + 8(13) = 0 + 108 + 104 = 212 \equiv 4 = \mathrm{E}$$

$$Decrypted\ plaintext: THE$$

Program (112550081_3.py):

```python
import numpy as np
import math

K = np.array([[6,24,1],[13,16,10],[20,17,15]])

def mod_inverse(a, m=26):
    for x in range(1, m):
        if (a * x) % m == 1:
            return x
    return None

def matrix_mod_inverse(K, m=26):
    # Step 1: Compute determinant
    det = int(round(np.linalg.det(K))) % m
    print(f"det(K) mod 26 = {det}")

    # Step 2: Verify gcd(det, 26) = 1
    assert math.gcd(det, m) == 1, "Matrix not invertible mod 26!"
    print(f"gcd({det}, 26) = 1 → Valid ✓")

    # Step 3: Modular inverse of determinant
    det_inv = mod_inverse(det, m)
    print(f"det_inv = {det_inv}")

    # Step 4: Compute adjugate matrix
    n = K.shape[0]
    cofactors = np.zeros((n, n), dtype=int)
    for i in range(n):
        for j in range(n):
            minor = np.delete(np.delete(K, i, axis=0), j, axis=1)
            cofactors[i][j] = ((-1)**(i+j)) * int(round(np.linalg.det(minor)))
    adj = cofactors.T
    print(f"Adjugate:\n{adj}")

    # Step 5: K_inv = det_inv * adj mod 26
    K_inv = (det_inv * adj) % m
    print(f"K_inv mod 26:\n{K_inv}")
    return K_inv

K_inv = matrix_mod_inverse(K)

# Step 6: Verify
assert np.array_equal(np.dot(K, K_inv) % 26,
                      np.eye(3, dtype=int)), "Verification failed!"
print("K × K_inv = I mod 26 ✓")

# Step 7: Decrypt AJN
c = np.array([0, 9, 13])  # A=0, J=9, N=13
p = np.dot(K_inv, c) % 26
print("Decrypted:", ''.join(chr(int(x)+65) for x in p))
```

```
det(K) mod 26 = 25
gcd(25, 26) = 1 → Valid ✓
det_inv = 25
Adjugate:
[[  70 -343  224]
 [   5   70  -47]
 [ -99  378 -216]]
K_inv mod 26:
[[ 8  5 10]
 [21  8 21]
 [21 12  8]]
K × K_inv = I mod 26 ✓
Decrypted: THE
```

Bonus Question.

Keys Used in Dach Stage

Stage	Cipher	Key
1	Affine Substitution	a=7, b=10
2	Columnar Transposition	Column order = [2, 0, 1] (3 columns)
3	Hill Cipher	$K = \begin{pmatrix} 6 & 24 & 1 \\ 13 & 16 & 10 \\ 20 & 17 & 15 \end{pmatrix}$

<u>Mathematical Formulation</u>

Stage 1: Affine Cipher

$$Encrypt: C \equiv 7P + 10 \ (mod\ 26)$$

$$Decrypt: P \equiv 15\ (C - 10)\ (mod\ 26)\ (since\ 7^{-1} \equiv 15\ (mod\ 26))$$

Stage 2: Columnar Transposition

Write plaintext into rows of 3 columns, then read columns in the order defined by key [2, 0, 1] (i.e., read column 2 first, then column 0, then column 1). Decrypt by reversing the column reordering.

Stage 3: Hill Cipher

$$Encrypt: c \equiv \mathrm{K} \cdot \mathrm{p}\ (\mathrm{mod}\ \ 26)$$

$$\text{Decrypt: } \mathrm{p} \equiv K^{-1} \cdot \mathrm{c} \pmod{26}, K^{-1} \equiv (\det K)^{-1} \cdot \mathrm{adj}\ (\mathrm{K}) \pmod{26}$$

<u>Worked Example with Plaintext: “CRYPTOGRAPHY”</u>

Stage 1: Affine Encryption $(f(x) = 7x + 10 \ \ mod\ 26)$:

Plaintext	C	R	Y	P	T	O	G	R	A	P	H	Y
P value	2	17	24	15	19	14	6	17	0	15	7	24
mod 26	24	25	22	11	13	14	0	25	10	11	7	22
Cyphertext	Y	Z	W	L	N	E	A	Z	K	L	H	W

After stage 1: YZWLNEAZKLHW

Stage 2: Columnar Transposition (key = [2, 0, 1])

Input: YZWLNEAZKLHW (12 letters, 3 columns -> 4 rows, no padding needed)

Step 1: write into grid row by row

	Col 0	Col 1	Col 2
Row 0	Y	Z	W
Row 1	L	N	E
Row 2	A	Z	K
Row 3	L	H	W

Step 2: read columns in key order [2, 0, 1], which means the column that gets rank 0 (read first) is col 1, rank 1 is col 2, rank 2 is col 3. So the output might be:

$$YZWLNEAZKLHW \rightarrow ZNZHWEKWYLAL$$

After stage 2: ZNZHWEKWYLAL

Stage 3: Hill Cipher: $c = Kp\ mod\ 26$, split “ZNZHWEKWYLAL” into 4 blocks of 3:

$$Block\ 1: ZNZ, Block\ 2: HWE, Block\ 3: KWY, Block\ 4: LAL$$

Block 1: Z=25, N=13, Z=25

$$\text{Row 1: } 6(25) + 24(13) + 1(25) = 150 + 312 + 25 = 487 \equiv 19 = \mathrm{T}$$

$$\text{Row 2: } 13(25) + 16(13) + 10(25) = 325 + 208 + 250 = 783 \equiv 3 = \mathrm{D}$$

$$\text{Row 3: } 20(25) + 17(13) + 15(25) = 500 + 221 + 375 = 1096 \equiv 4 = \mathrm{E}$$

Block 2: H=7, W=22, E=4

$$\text{Row 1: } 6(7) + 24(22) + 1(4) = 42 + 528 + 4 = 574 \equiv 2 = \mathrm{C}$$

$$\text{Row 2: } 13(7) + 16(22) + 10(4) = 91 + 352 + 40 = 483 \equiv 15 = \text{P}$$

$$\text{Row 3: } 20(7) + 17(22) + 15(4) = 140 + 374 + 60 = 574 \equiv 2 = \text{C}$$

Block 3: K=10, W=22, Y=24

$$\text{Row 1: } 6(10) + 24(22) + 1(24) = 60 + 528 + 24 = 612 \equiv 14 = \text{O}$$

$$\text{Row 2: } 13(10) + 16(22) + 10(24) = 130 + 352 + 240 = 722 \equiv 20 = \text{U}$$

$$\text{Row 3: } 20(10) + 17(22) + 15(24) = 200 + 374 + 360 = 934 \equiv 24 = \text{Y}$$

Block 3: K=10, W=22, Y=24

$$\text{Row 1: } 6(11) + 24(0) + 1(11) = 66 + 0 + 11 = 77 \equiv 25 = \text{Z}$$

$$\text{Row 2: } 13(11) + 16(0) + 10(11) = 143 + 0 + 110 = 253 \equiv 19 = \text{T}$$

$$\text{Row 3: } 20(11) + 17(0) + 15(11) = 220 + 0 + 165 = 385 \equiv 21 = \text{V}$$

$$\text{ZNZHWEKWYLAL} \rightarrow \text{TDECPCOUYZTV}$$

After stage 3: TDECPCOUYZTV, so the final Ciphertext is “TDECPCOUYZTV”

We could also check the result of implementing code below, for the code please check 112550081_bonus.py for further details:

```
==================================================
           ENCRYPTION PROCESS
==================================================
Original Plaintext     : CRYPTOGRAPHY

[Stage 1] Affine Cipher  (a=7, b=10)
  Formula  : C = (7P + 10) mod 26
  Output   : YZWLNEAZKLHW

[Stage 2] Columnar Transposition  (key=[2, 0, 1])
  Formula  : write into rows, read columns by key order
  Output   : ZNZHWEKWYLAL

[Stage 3] Hill Cipher  (3x3 key matrix K)
  Formula  : c = K * p  (mod 26)
  Output   : TDECPCOUYZTV

Final Ciphertext       : TDECPCOUYZTV

==================================================
           DECRYPTION PROCESS
==================================================
[Stage 3 inv] Hill Decrypt
  Formula  : p = K^-1 * c  (mod 26)
  Output   : ZNZHWEKWYLAL

[Stage 2 inv] Transposition Decrypt
  Formula  : reverse column reordering
  Output   : YZWLNEAZKLHW

[Stage 1 inv] Affine Decrypt
  Formula  : P = 15*(C - 10)  mod 26
  Output   : CRYPTOGRAPHY

Recovered Plaintext    : CRYPTOGRAPHY
Original  Plaintext    : CRYPTOGRAPHY
Verification           : PASS ✓
```